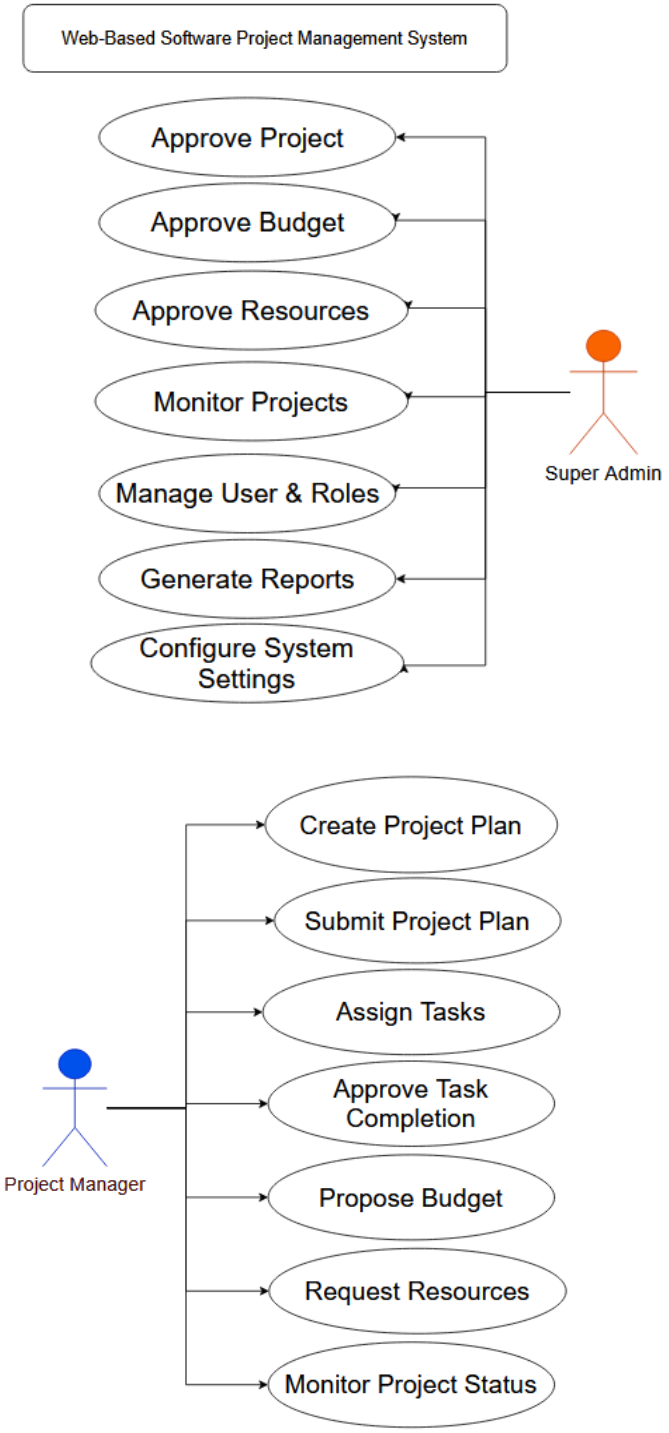
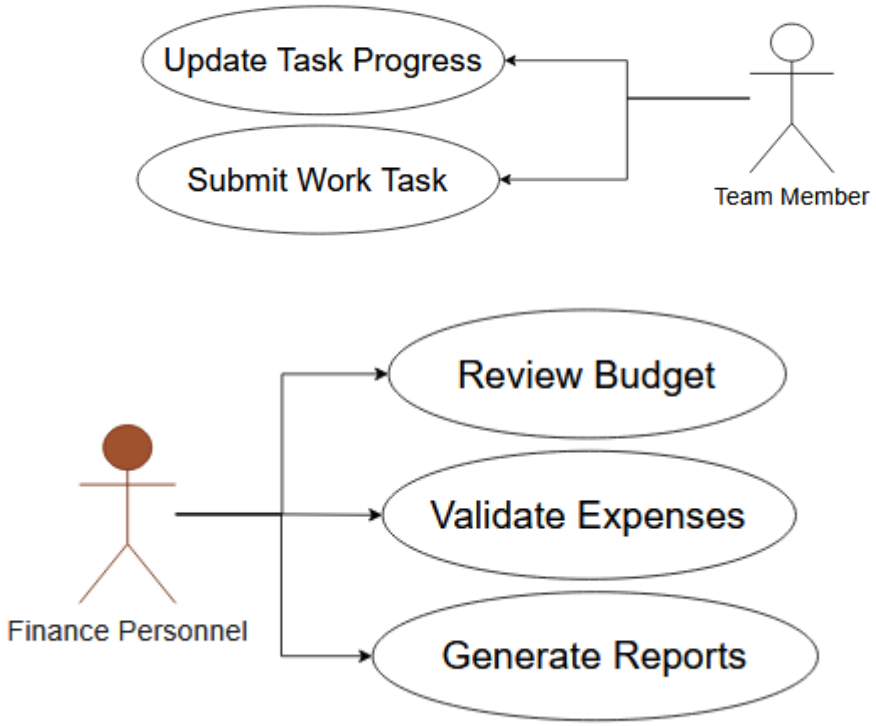
FINAL PROJECT DOCUMENTATION

NAME	**CHARLES JUNE A. JOAQUIN**
PROJECT TITLE	**CSharpCharles Project Management System**
SUBJECT: **CODE:** **TIME:**	**IT15/L Integrative Programming and Technologies** **8441** **10:00AM-12:00PM**
TOPIC (Type of Business Process)	**#16 Project Management System**
Products/Services	**Project Management and Financial Tracking**
Website/Deployed (Link)	**http://csharpcharles.runasp.net/**
API/ALGO/MODEL:	MicrosoftWeb API, GoogleCalendar API, Job Salary Data API
Security Features	User Authentication, Role-Based Access Control, Input Validation, Activity Logging, SSL/HTTPS Encryption, Microsoft Graph API, Google Calendar API, QuickBooks API.
Target user/s:	Project Manager, Software Team Member (Developer / Tester), Finance Personnel, and System Administrator.
Subs System / Management Transaction/Modules	1.Project Planning 2.Task Scheduling 3.Resources Allocation 4. Project Monitoring 5.Budget and Income
LOGO	
Project Objectives:	• Develop a web-based ERP system that integrates project planning, task scheduling, resource allocation, project monitoring, and budget management into a unified platform accessible across the organization.

	● Provide an integrated project planning and task scheduling module that enables users to create projects, define timelines, assign tasks, and track progress for improved coordination and on-time delivery.. ● Implement a resource management module to efficiently allocate, monitor, and optimize the utilization of human resources, equipment, and materials across multiple projects. ● Enable real-time project monitoring and performance tracking to provide managers and stakeholders with up-to-date project status, milestones, and key performance indicators for data-driven decision-making. ● Integrate a financial management module to manage budgets, track expenses and income, and provide financial oversight, ensuring cost control and strategic resource planning across all projects.
Project Description:	The CSharpCharles Project Management System is a web-based ERP application designed to centralize and integrate project management activities for software development organizations. The system targets Project Managers, Team Members, Finance Officers, and Super Admins, each with specific responsibilities and access levels. Users expect a platform that enables efficient project planning, task assignment, resource allocation, progress tracking, and financial management, while providing clear workflows and approval processes for accountability and governance. By consolidating project data and automating workflows, the system is expected to positively impact users and the organization by improving coordination, transparency, and efficiency. Teams can track progress in real time, managers can optimize resource utilization, and Finance Officers can validate budgets accurately. Overall, the ERP approach reduces manual work, minimizes errors, enhances collaboration across departments, and ensures timely project delivery, providing tangible benefits to both the organization and stakeholders. The system will be developed using C# and ASP.NET for backend development, HTML, CSS, and JavaScript for the frontend, and SQL Server as the database management system. Visual Studio and SQL Server Management Studio (SSMS) will be used for development and database management. This stack supports enterprise-grade web applications and provides the tools necessary to implement integrated ERP modules for project, resource, and financial management. These technologies were chosen due to their stability, security, scalability, and compatibility with enterprise-level applications. C# and ASP.NET allow structured implementation of project management workflows and approval systems, while SQL Server ensures data integrity and efficient storage of project, task, resource, and financial information. The web-based architecture ensures accessibility across devices and departments, aligning perfectly with the system's functional and technical requirements for a centralized, ERP-style project management platform.
Type of Users/ Role-Based Access	**1. System Administrator (Suer Admin)** Username:superadmin@CSharpCharles.com Password:admin123 **2. Project Manager** Username: charizut@CSharpCharles.com Password:password123

	3. Finance Officer Username:jarizut@CSharpCharles.com Password:password123 **4. Employee** Username:Jeriel@CSharpCharles.com Password:jerie123
Use Case Diagram (Role-Based Access)	



Data Dictionary

CSharpCharles Project Management System

Users table

Field Names	Datatype	Length	Description
UserID-PK	Int-AI	9	Unique User ID Number
FullName	Text	50	User's Complete Name
Email	Text	50	User's email address
UserName	Text	30	User's Name or email
Password	Text	50	User's Login Password
Role	Text	25	User Role

Projects Table

Field Names	Datatype	Length	Description
ProjectID-PK	Int-AI	9	Unique project ID number

ProjectName	Text	50	Name/Title of the project
Description	Text	200	Project overview/details
Status	Text	25	Project Status (Pending, Approved, In progress, Completed)
StartDate	Date	-	Project State Date
EndDate	Date	-	Project expected completion date

Tasks Table

Field Names	Datatype	Length	Description
TaskID-PK	Int-AI	9	Unique Task ID Number
TaskName	Text	50	Task title/name
Description	Text	200	Task details
Status	Text	25	Task Progress (Not started, In Progress, Completed)
Priority	Text	15	Task Priority Level(Low, Medium, High)
DueDate	Date	-	Deadline of the Task

Budgets Table

Field Names	Datatype	Length	Description
BudgetID-PK	Int-AI	9	Unique Budget ID Number
Amoint	Decimal	10,2	Total Proposed budget amount
status	Text	25	Budget status (pending, Approved, Rejected)
Date Proposed	Date	-	Date budget was proposed

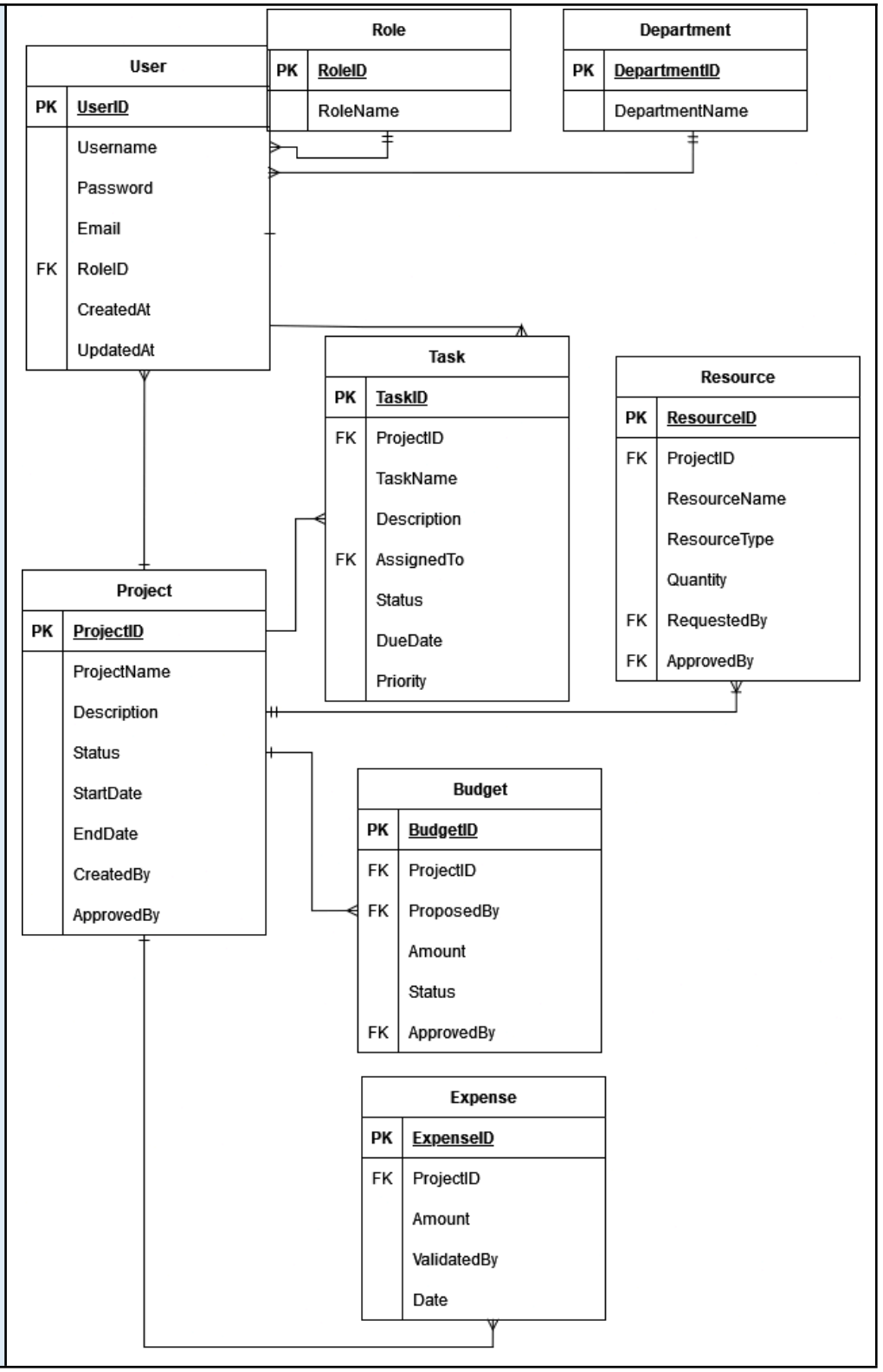
Resources Table

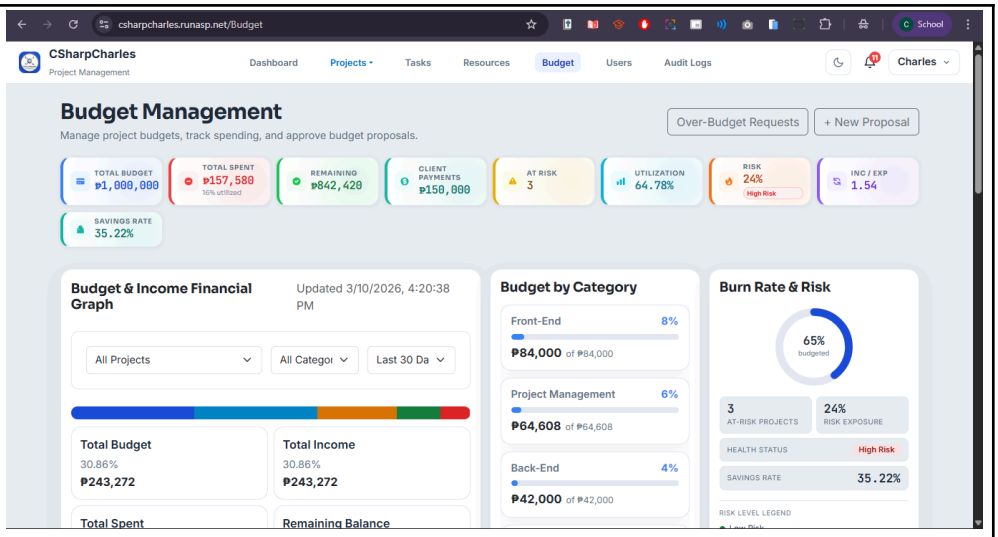
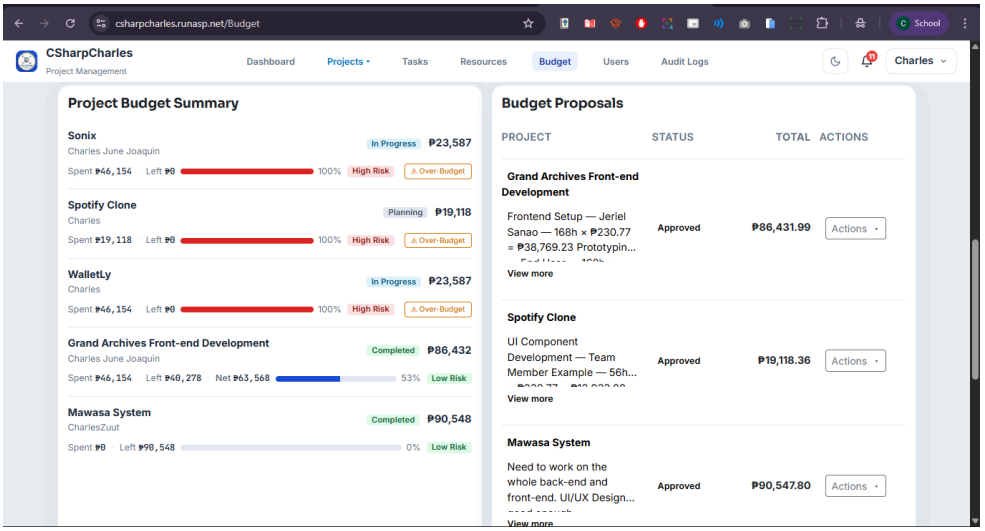
Field Names	Datatype	Length	Description
ResourceID-PK	Int-AI	9	Unique resource ID number
ResourceName	Text	50	Name of resource (Equipment, Staff, Material)
ResourceType	Text	25	Type of Resource(Human Equipment, Material)
Quantity	Int	9	Number of resources allocated
Status	Text	25	Resource status (Requested, Approved Allocated)

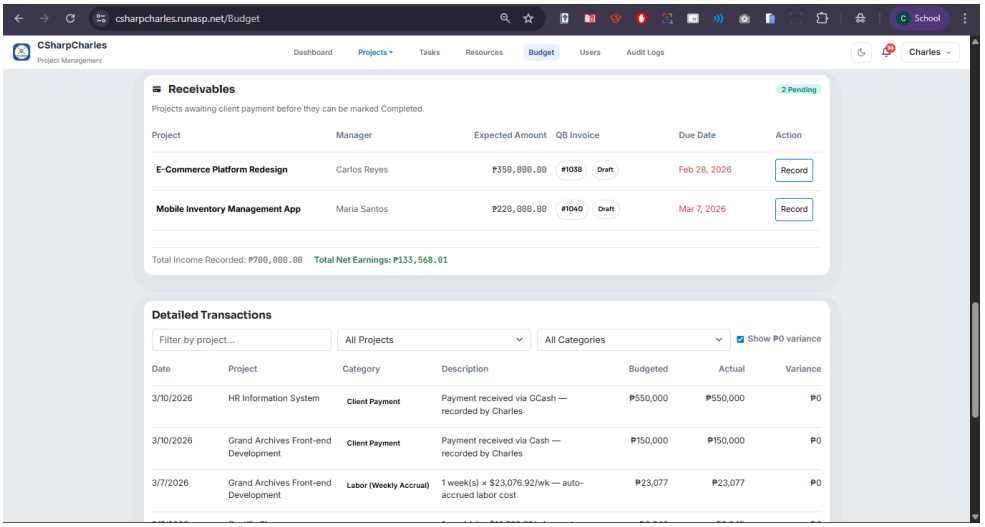
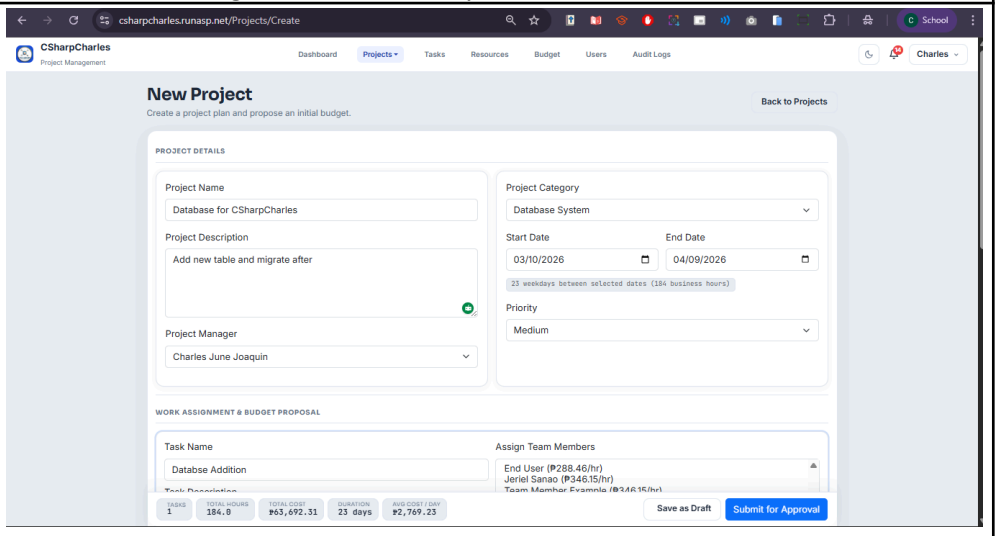
Expenses Table

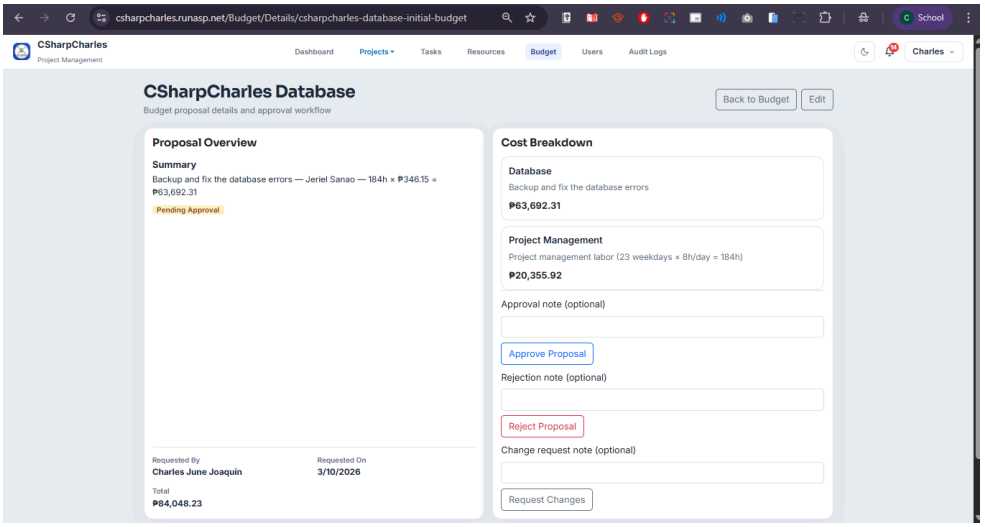
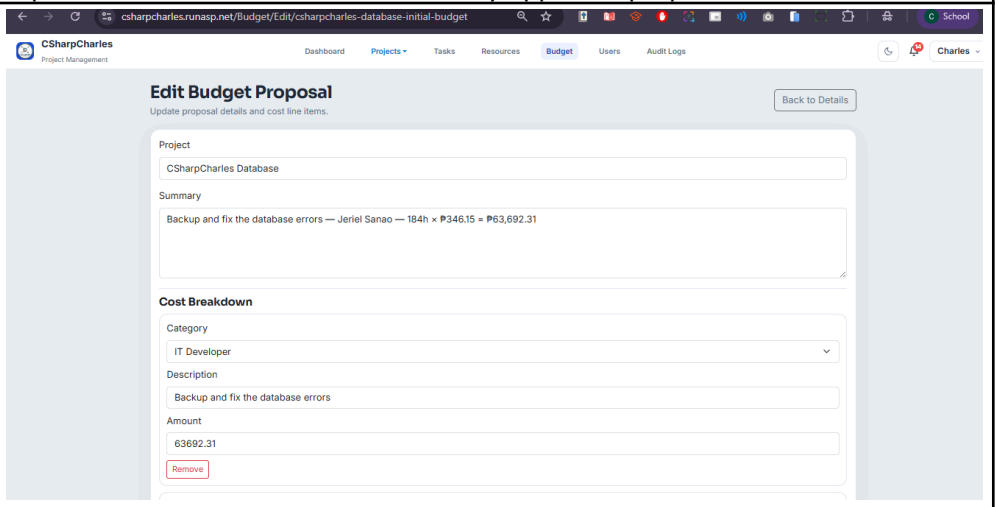
Field Names	Datatype	Length	Description
EmpID-PK	Int-AI	9	Employees ’s ID Number
UserID-FK	Int	9	User’s ID Number
EmpNumber	Text	25	Employees ’s Company Number
LastName	Text	20	Employees ’s Last Name
FirstName	Text	30	Employees ’s First Name
MiddleName	Text	20	Employees ’s Middle Name

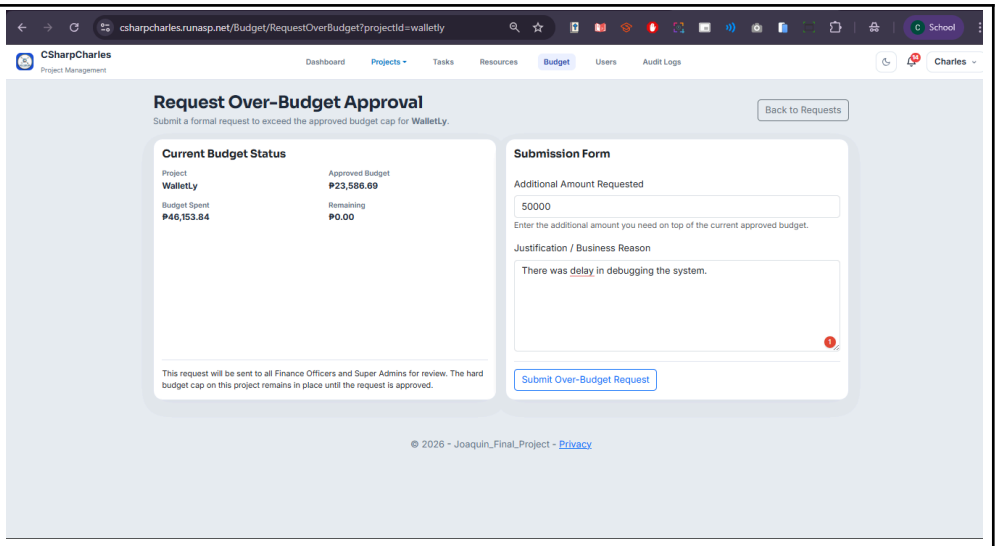
Entity Relational Diagram (ERD)



Prototype (Frontend)	 **Budget Management Dashboard** The main financial hub of the system. The page opens with a 9-tile KPI strip displaying Total Budget, Total Spent, Remaining, Client Payments, At-Risk count, Utilization %, Risk %, Inc/Exp Ratio, and Savings Rate — all in Philippine Peso (₱). Below the KPIs is a 3-column bento layout containing the Budget & Income Financial Graph (interactive stacked bar chart with Project/Category/Time-range filters), Budget by Category (horizontal progress cards), and Burn Rate & Risk (donut gauge). Users can filter the financial graph using the dropdowns to narrow by project, category, or time window (Last 30 Days, 90 Days, etc.) and watch all figures update in real time (auto-refresh every 20 seconds).
	 **Project Budget Summar & Budget Proposals** The second row contains two panels. The left panel displays a project budget card listing each project with its allocation, amount spent, remaining balance, net earnings (if applicable), a color-coded utilization bar, and a risk badge (Low / Moderate / High). Projects at 90%+ utilization surface a red ⚠ Over-Budget button for SuperAdmins and Project Managers, linking to that project's over-budget request form. The right panel lists all budget proposals in a table (desktop) or card layout (mobile), showing each proposal's project name, status badge, total amount, and an Actions menu. Status badges: Draft (grey), Pending (yellow), Changes Requested (blue), Approved (green), Rejected (red). SuperAdmins and Finance

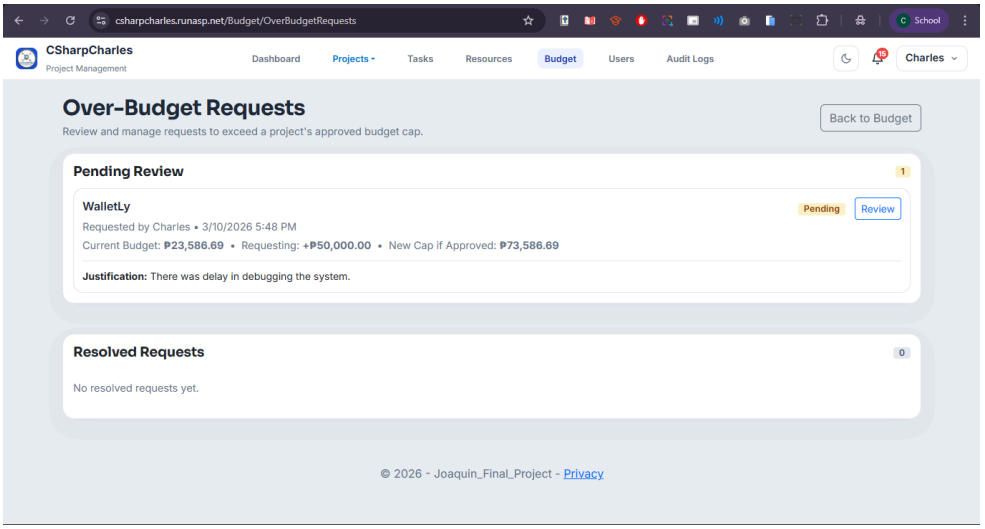
	Officers can approve, reject, or request changes via the Actions dropdown; Project Managers can view or edit unapproved drafts.
	 **Receivables & Detailed Transactions** The third row tracks projects awaiting client payment before closure. Each entry shows the project name, manager, expected payment amount, QuickBooks invoice number and status (Draft / Sent / Paid / Void), and due date (highlighted red if overdue). A Record button links to the Project Details page to log incoming payments, and Finance staff can generate a QuickBooks invoice directly from the table if none exists. Total Income Recorded and Total Net Earnings appear as a footer summary. Spanning the full width at the bottom of the Budget Index is the primary transaction ledger, displaying every cost line item across all projects with columns for Date, Project, Category, Description, Budgeted, Actual, and Variance — negative variances shown in red. Records can be filtered by free-text project name search, a Project dropdown, a Category dropdown (e.g., IT Developer, Software Developer), and a toggle to show or hide zero-variance rows. Results are paginated at 10 per page with Prev/Next navigation, making this the central screen for reviewing all financial activity.
	 **Create new Budget Proposal** A form to create a formal budget proposal for a project. Fields include Project Name, Summary/notes (optional textarea), and a dynamic Cost Breakdown section where users add line items — each with a Category dropdown (IT

	Developer, Software Developer, Software Engineer, Project Manager, Other), Description, and Amount. Additional rows can be added with the "+ Add Item" button; rows can be removed individually. A live summary at the bottom shows the running Status and Total Proposed amount. Submitting saves a Draft proposal that can later be submitted for approval.
	 **Budget Proposal Details** The read-only detail page for a single budget proposal. The left card shows the Proposal Overview: Summary, current Status badge, Active Budget indicator, Requested By, Requested On (PHT), Total Amount, and any Decision information (Approved/Rejected by, date, decision note). The right card shows the full Cost Breakdown table with each line item (Category, Description, Individual Amount, Subtotals per category). SuperAdmins and PMs can navigate to Edit from here; SuperAdmins can also override an already-approved proposal.
	 **Edit Budget Proposal** Identical in layout to the Create form but pre-populated with existing data. Users can update the Project Name, Summary, and all cost line items — add new rows or remove existing ones. The live Total Proposed updates instantly as amounts are changed. Saving the form updates the proposal and returns the user to the Proposal Details page. This screen is also used by SuperAdmins to override an approved budget when exceptional circumstances require it.



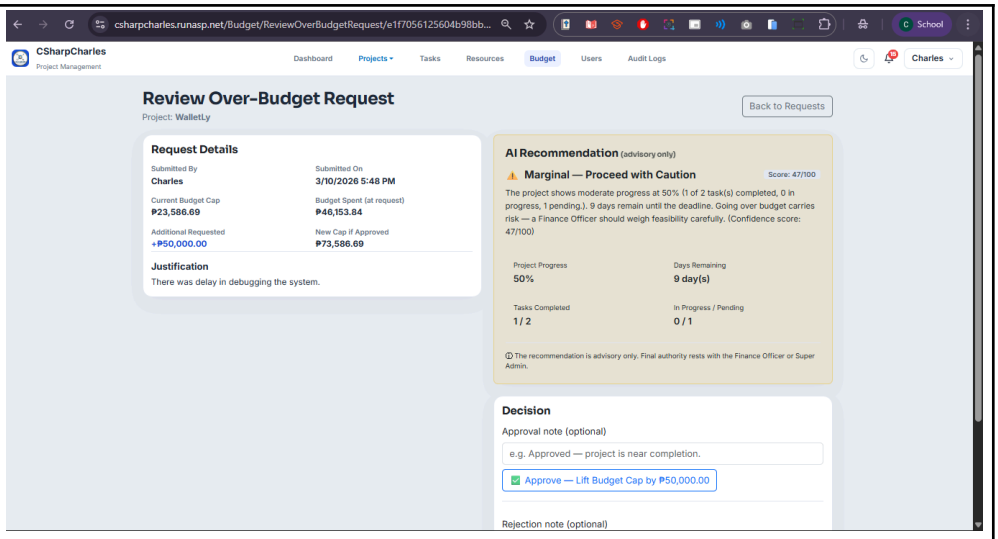
Submit Over-Budget Request

A two-column page for formally requesting additional budget for a specific project. The left card shows the Current Budget Status: project name, approved budget, amount already spent, and remaining balance — giving context for why the request is needed. The right card is the Submission Form with two fields: Additional Amount Requested (numeric, in ₱) and a Justification textarea. Submitted requests are routed to all Finance Officers and Super Admins for review. The hard budget cap on the project remains locked until the request is approved.



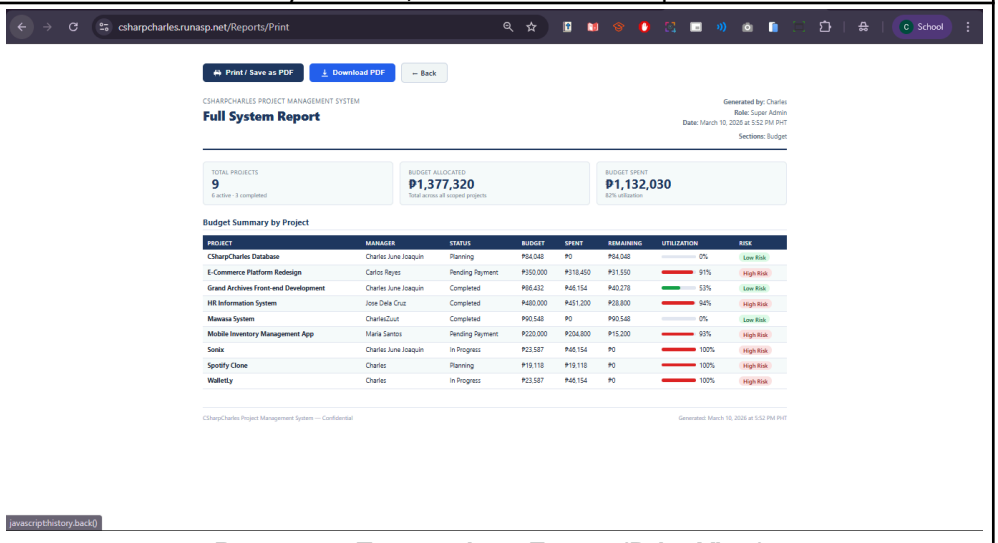
Over-Budget Requests List

A management view split into two sections: Pending Review (with a yellow count badge) and Resolved Requests. Each pending request card shows the project name, who submitted it, when it was submitted (PHT timestamp), the current budget cap, the additional amount being requested, and the projected new cap if approved. A Review button leads to the detailed review page. The Resolved section shows historical requests with their final Approved/Rejected status, reviewer name, and review date.



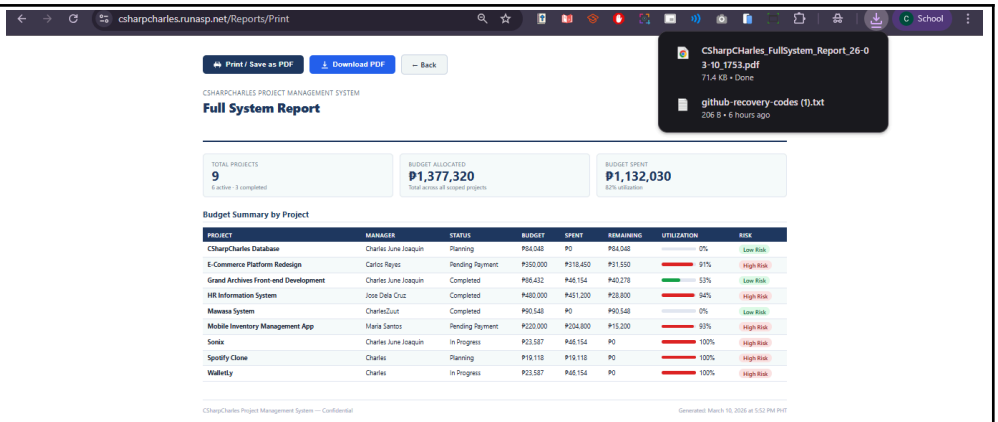
Review Over-Budget Request

The decision page for an over-budget request. The left panel shows Request Details: submitted by, submitted date/time (PHT), current budget cap, budget spent at the time of request, additional amount requested, and the justification text. The right panel shows an AI Recommendation panel with a color-coded verdict (green = Worth It / yellow = Marginal / red = Not Worth It), a confidence score, and a grid of supporting signals (e.g., project completion %, current burn rate, risk level). The reviewer can then Approve or Reject the request using the action buttons. If already resolved, a banner shows the previous decision.



Reports — Transactions Export (Print View)

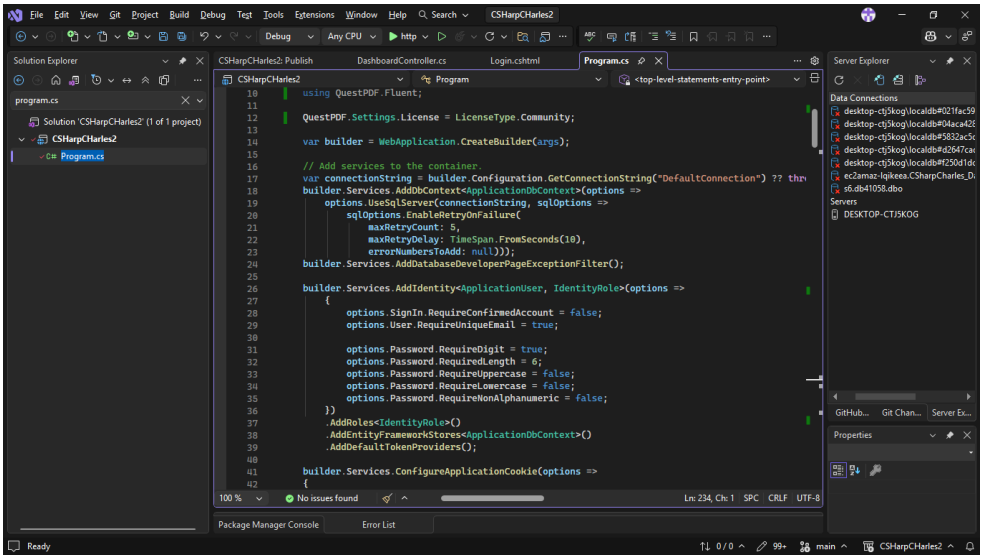
The formal report generation screen. Select Transactions from the Report Type dropdown (or Full System to include all sections), then click Generate Report. The page renders a structured report with a header (generated by, role, PHT timestamp), a KPI summary row, and a Detailed Transactions table with columns: Date | Project | Category | Description | Amount. Users can print directly to PDF via the browser using Print / Save as PDF, or download a properly formatted A4 PDF file using the Download PDF button.



Downloaded PDF — Transactions Section

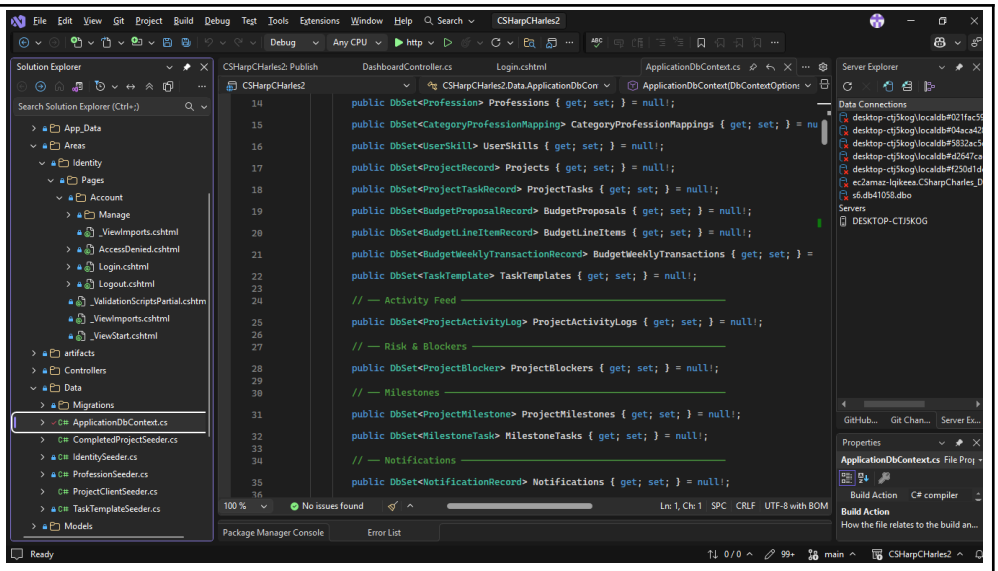
The exported QuestPDF document opens as a downloaded .pdf file named in the format CSharpCHarles_{ReportType}_Report_{YY-MM-DD}_{HHmm}.pdf (e.g., CSharpCHarles_Transactions_Report_25-06-15_1430.pdf). The PDF has a navy-header style, displays the report title, generator name, role, and PHT generation time in the header, followed by the transactions table. Each page is A4 size with proper margins. Take a screenshot of the opened PDF to show the final polished output.

Prototype (Backend)



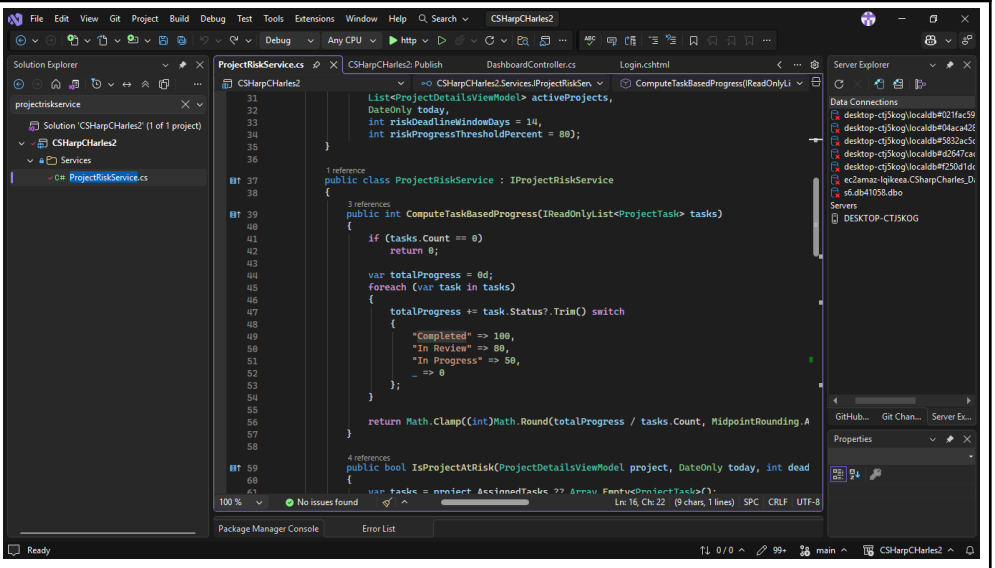
Application Startup and service registration

Configures the entire DI container: EF Core with SQL Server retry logic (5 retries, 10s delay), ASP.NET Core Identity with role support, Google OAuth 2.0 with calendar scope (auto-capturing tokens into OAuthTokenStore on login), typed HttpClient instances for Google Calendar, Microsoft Graph, and QuickBooks APIs, rate limiting (60 req/IP/hour), and a Content-Security-Policy middleware. Five database seeders run on startup. APIs used: ASP.NET Core Identity, EF Core, Google OAuth 2.0, QuestPDF Community.



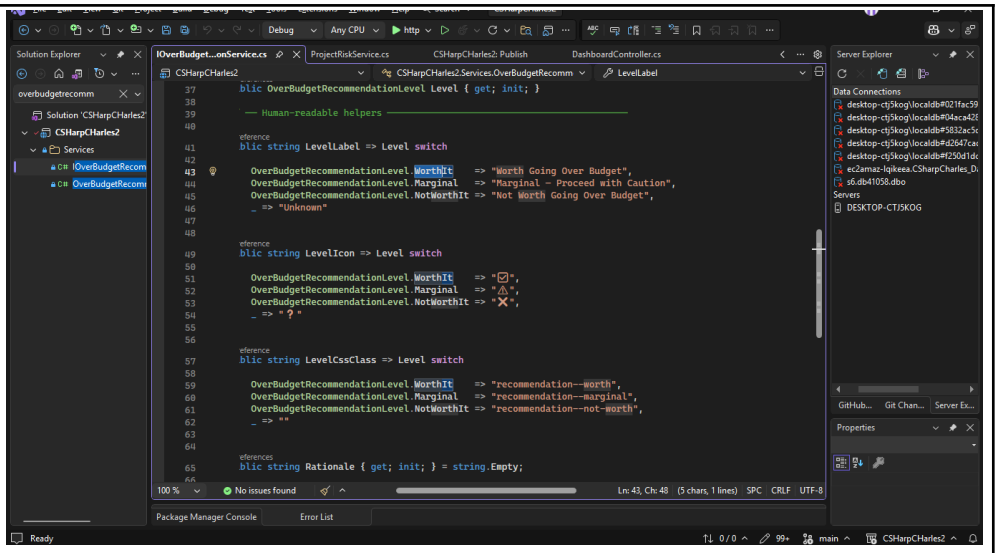
Database Context

Inherits from IdentityDbContext<ApplicationUser> and declares 22 DbSet properties covering projects, tasks, budget proposals, transactions, milestones, blockers, notifications, audit logs, OAuth tokens, portal links, and over-budget requests. Configures cascade delete rules, decimal precision, and composite database indexes for query performance. Algorithm: EF Core Fluent API relationship and index configuration.



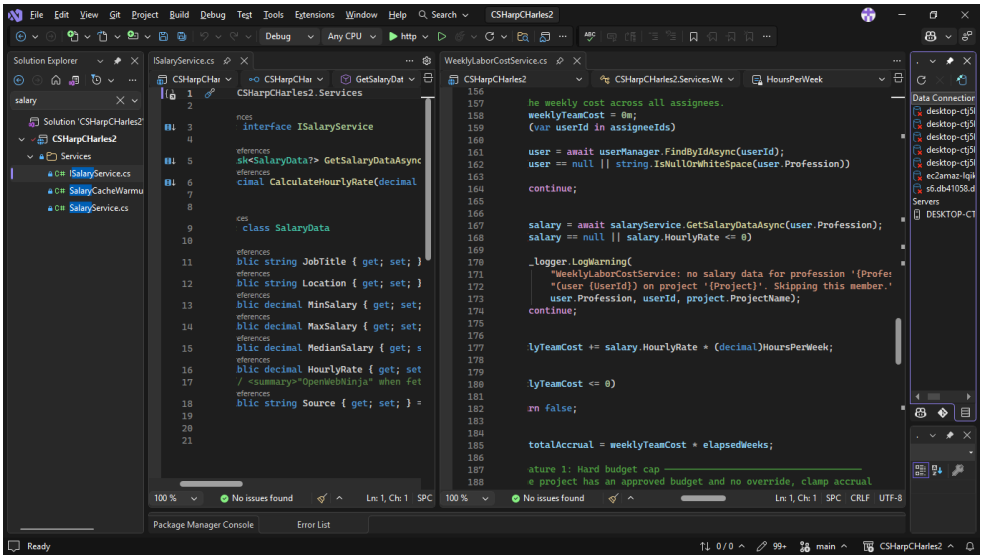
Project Risk Scoring

Computes task-based progress by averaging a weighted status map across all tasks (Completed=100, In Review=80, In Progress=50, else 0). Flags a project "At Risk" when progress is below 80% and the deadline is within 14 days. Used by both DashboardController and ReportsController. Algorithm: Weighted average progress scoring with deadline proximity threshold.



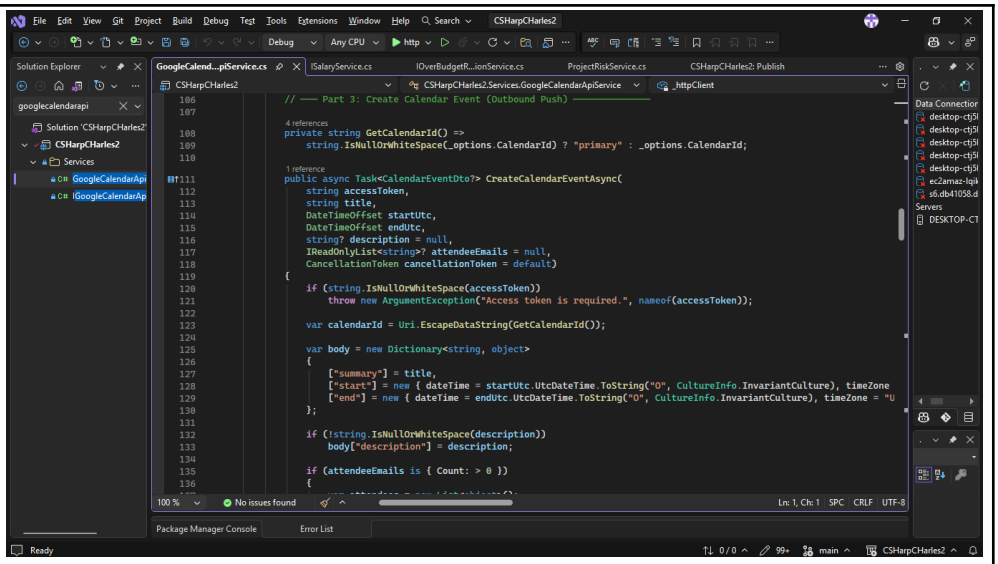
Over-budget AI Recommendation

Evaluates whether an over-budget request is worth approving using a 0–100 composite score: project progress (45%), task completion ratio (35%), and time urgency (20%). Scores ≥65 = "Worth It", ≥35 = "Marginal", below = "Not Worth It". Only activates when the deadline is within 14 days. Algorithm: Weighted composite scoring with configurable thresholds.



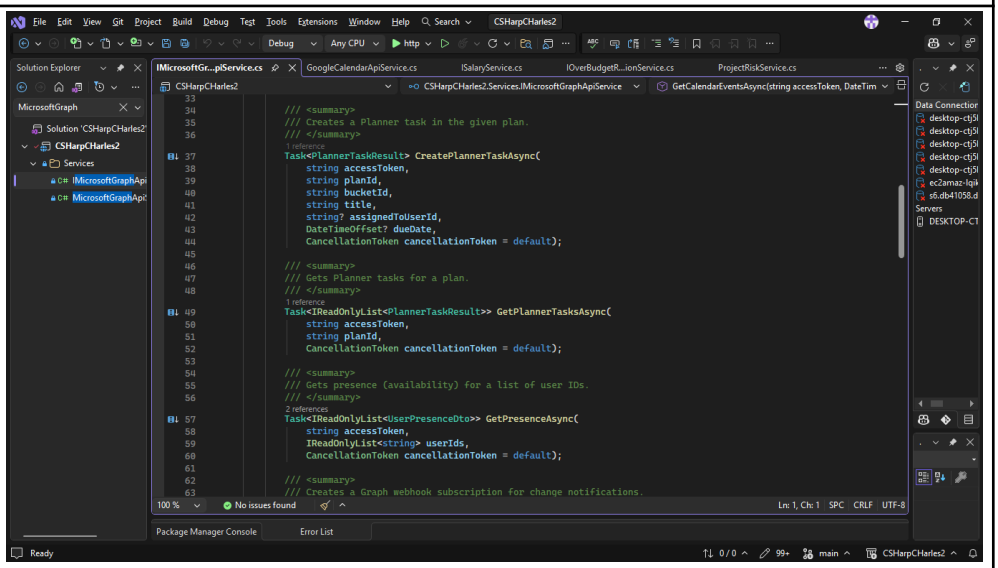
Salary Data & Labor Cost

SalaryService fetches real-time salary data from the OpenWebNinja Job Salary API, caches results in IMemoryCache for 24 hours, and falls back to hardcoded Philippine market ranges (e.g., Software Developer ₱420k–₱780k/yr) when unavailable. WeeklyLaborCostService uses those rates to accrue 40 hrs/week per task assignee onto each project's BudgetSpent, back-filling all elapsed weeks from the project start date, running every 7 days. APIs/Algorithms: OpenWebNinja API, IMemoryCache TTL, week-based cost accrual with historical back-fill.



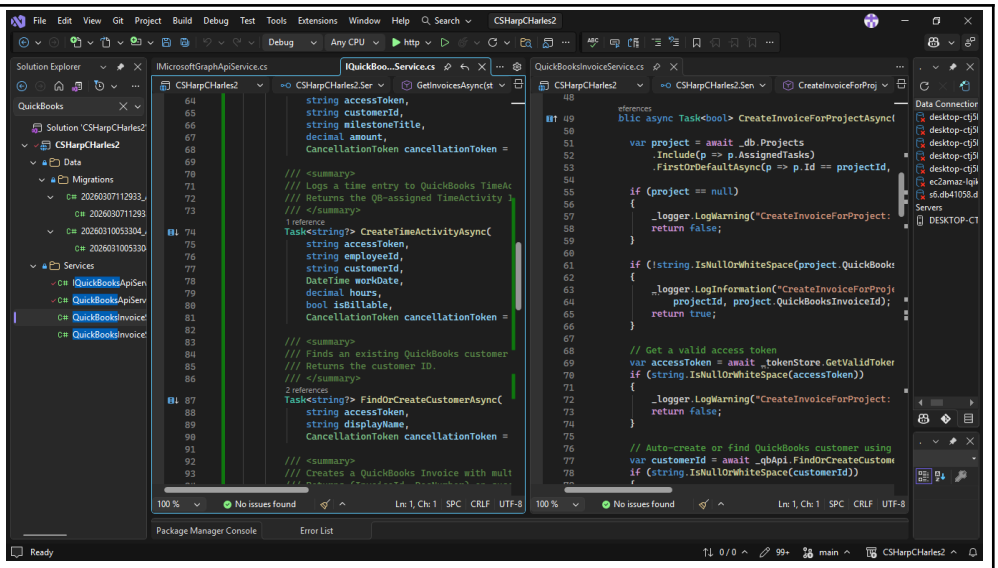
Google Calendar Integration

Calls the Google Calendar API v3 (/calendar/v3/calendars/{id}/events) with a Bearer token to retrieve up to 100 events in a given date range. OAuth tokens are retrieved from OAuthTokenStore and automatically refreshed before expiry via TokenRefreshBackgroundService. API used: Google Calendar REST API v3 with OAuth 2.0.



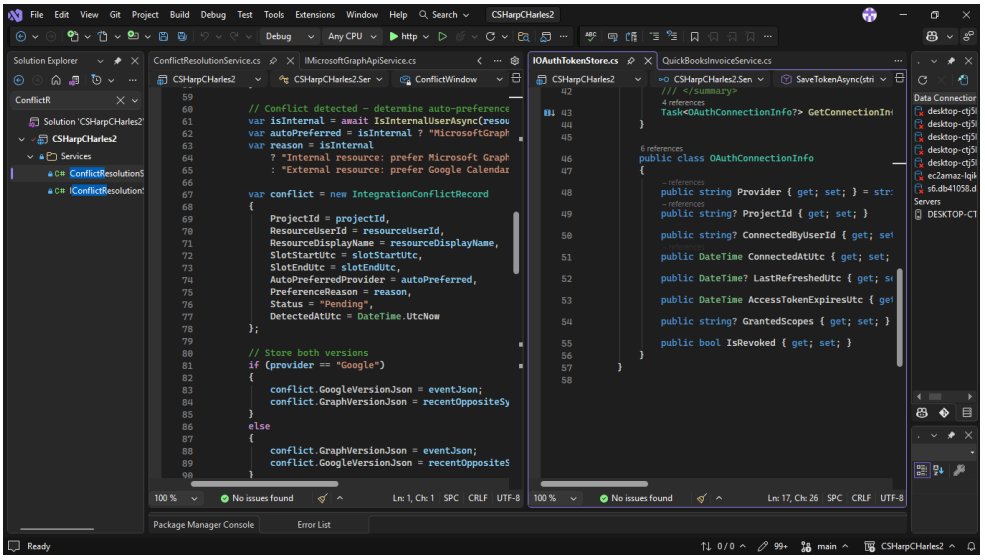
Microsoft Graph Integration

Connects the system to Microsoft 365 services via the Microsoft Graph API, enabling seamless synchronization of users, calendars, emails, and organizational data. Used to authenticate users through Azure Active Directory, sync meeting schedules, and pull profile information directly from the organization's Microsoft tenant — reducing manual data entry and keeping records consistent across platforms.



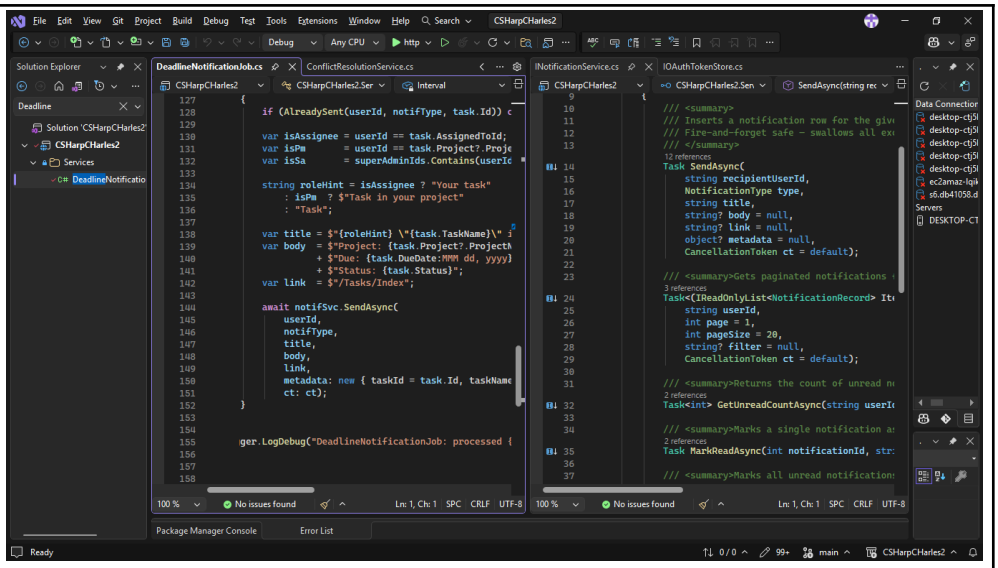
QuickBooks Integration

QuickBooksApiService queries the QuickBooks Online API (/v3/company/{id}/query) using IQL to retrieve invoices. QuickBooksInvoiceService auto-creates invoices when a project enters "Pending Payment" status and syncs invoice status — automatically completing the project when the QuickBooks invoice is marked Paid. QuickBooksInvoiceSyncJob runs this sync periodically in the background. API used: QuickBooks Online REST API v3 with OAuth 2.0.



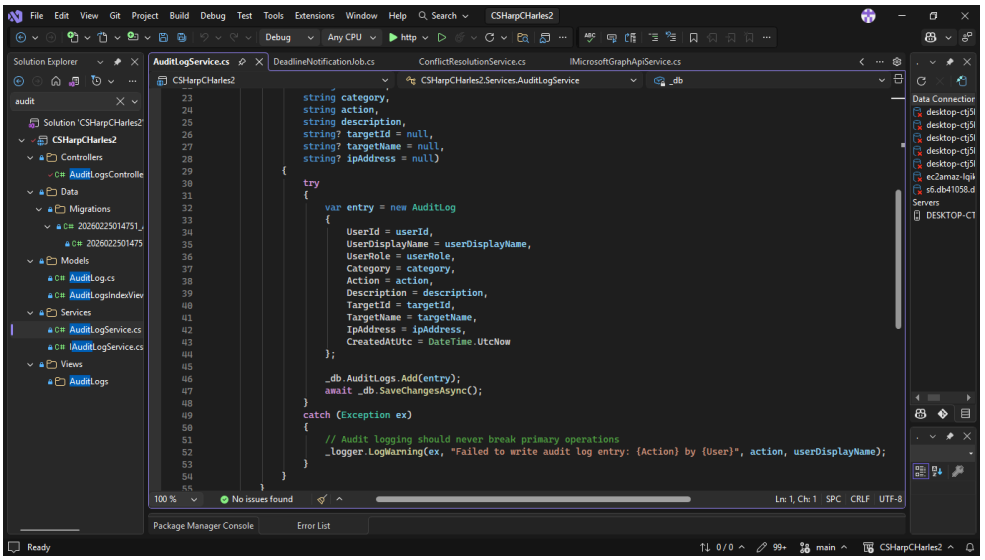
OAuth Token Store & Conflict Resolution

OAuthTokenStore persists encrypted OAuth tokens (using ASP.NET Core Data Protection) for Google, Microsoft Graph, and QuickBooks, and silently refreshes them within a 5-minute expiry buffer. ConflictResolutionService detects dual-provider calendar sync conflicts within a 30-second window and auto-resolves them by preferring the most recent provider. Algorithms: AES-256 Data Protection encryption, time-window conflict detection.



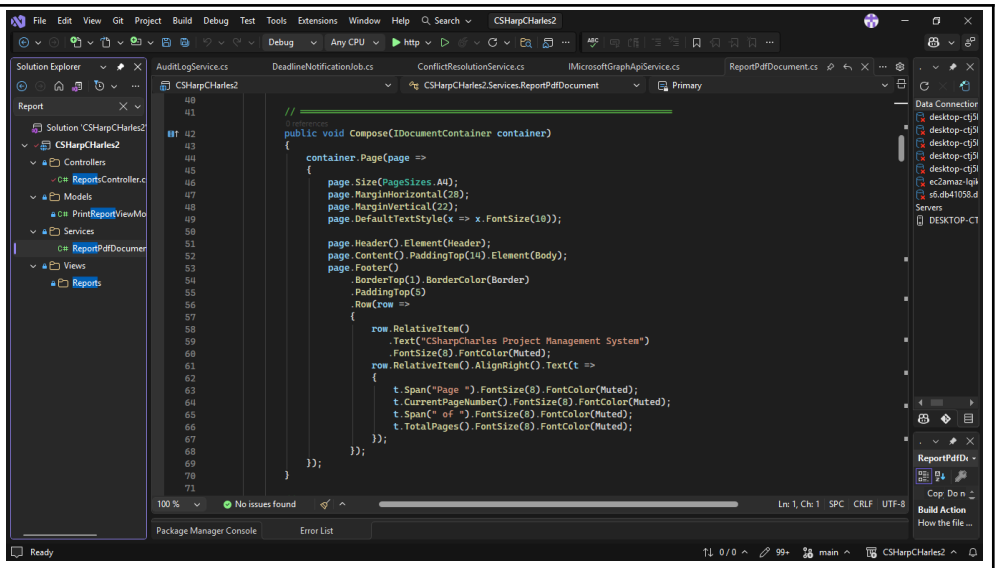
Notifications & Background Jobs

NotificationService persists typed in-app notifications (tasks, approvals, blockers, milestones, deadlines, budget, system) to the database. DeadlineNotificationJob scans tasks every 6 hours and fires notifications at 7, 3, and 1 day(s) before due date, deduplicated within 20 hours. CriticalTaskNotificationJob (hourly) sends Teams alerts for unstarted Critical-priority tasks 48+ hours after project start. SystemHealthJob (every 6 hours) performs predictive analysis — flagging overdue projects, budget overruns, and stalled projects — and notifies all SuperAdmins. Algorithm: Deduplication window checks, deadline proximity scanning.



Audit Logging

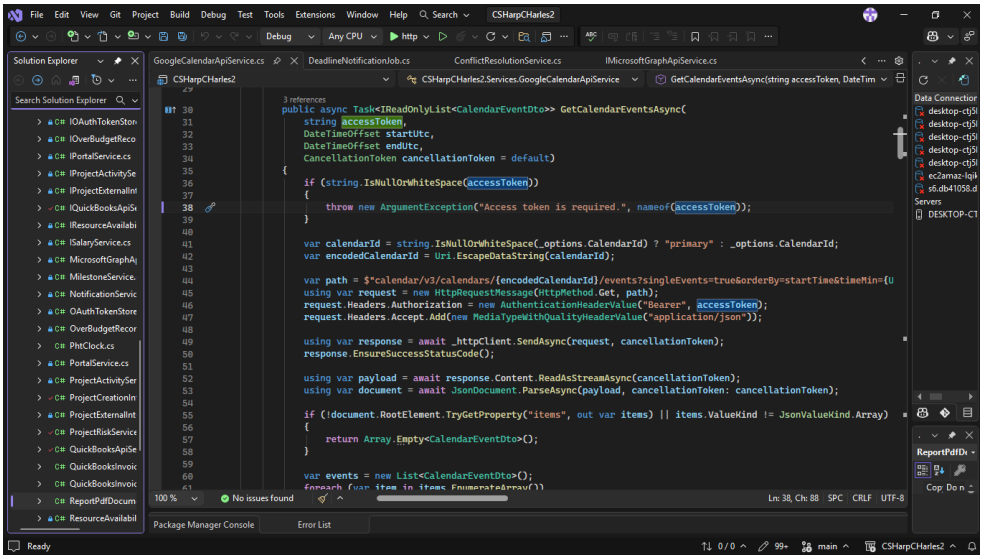
Records every privileged action (user, role, category, action, description, target, IP address, UTC timestamp) to the AuditLogs table. Failures are swallowed and logged as warnings so a logging error never disrupts a primary operation. Called from all major controllers — Budget, Projects, Reports, Users, and Integrations.



PDF Report Generation

Implements QuestPDF's IDocument interface to render a PrintReportViewModel into a formatted A4 PDF. Contains six section renderers (Projects, Budget, Members, Over-Budget, Transactions, Audit Log) with navy-header tables, alternating row shading, and color-coded risk badges. The Download action in ReportsController streams the PDF bytes directly to the browser as CSharpCHarles_{type}_Report_{yy-MM-dd}_{HHmm}.pdf. Library used: QuestPDF 2026.2.3 (Community License), fluent layout API.

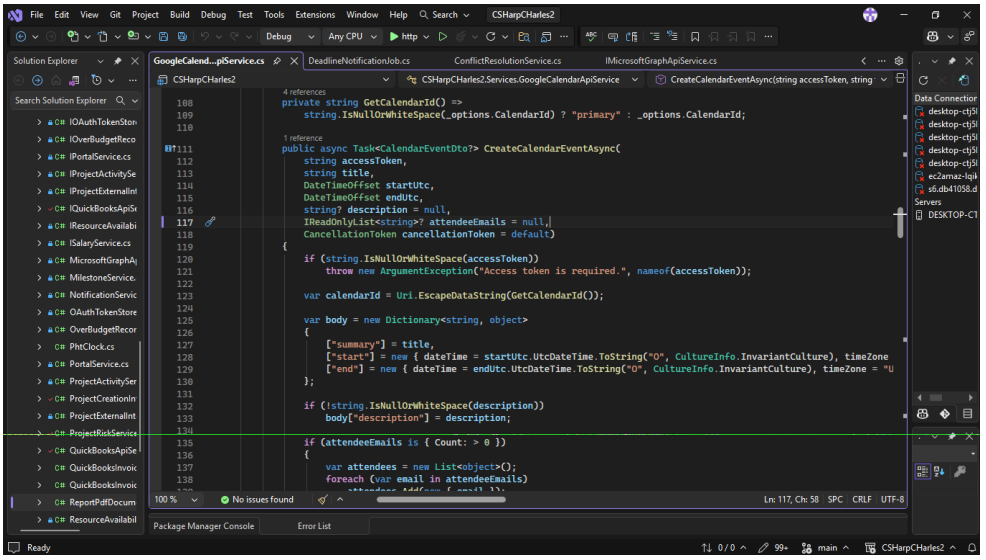
API FUNCTIONS/ FEATURES



Google Calendar API — Fetch Calendar Events

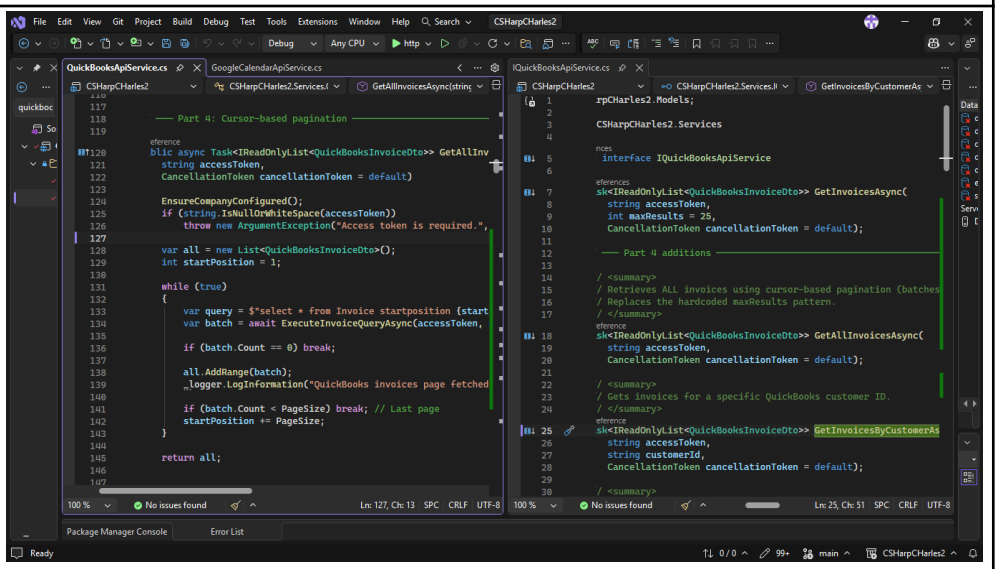
Calls GET /calendar/v3/calendars/{calendarId}/events on the Google Calendar API v3, passing timeMin, timeMax (ISO 8601 UTC), singleEvents=true, orderBy=startTime, and maxResults=100. A Bearer OAuth access token is attached to the Authorization header. The JSON response is streamed and parsed using System.Text.Json; each item in the items array is mapped to a CalendarEventDto containing the event id, summary (title), start/end times, organizer email, and htmlLink. The TryReadStartOrEnd helper handles both timed events (dateTime) and full-day events (date), preventing parse failures on all-day entries. Used by

DashboardController to display a team member's upcoming Google Calendar schedule alongside their project tasks.



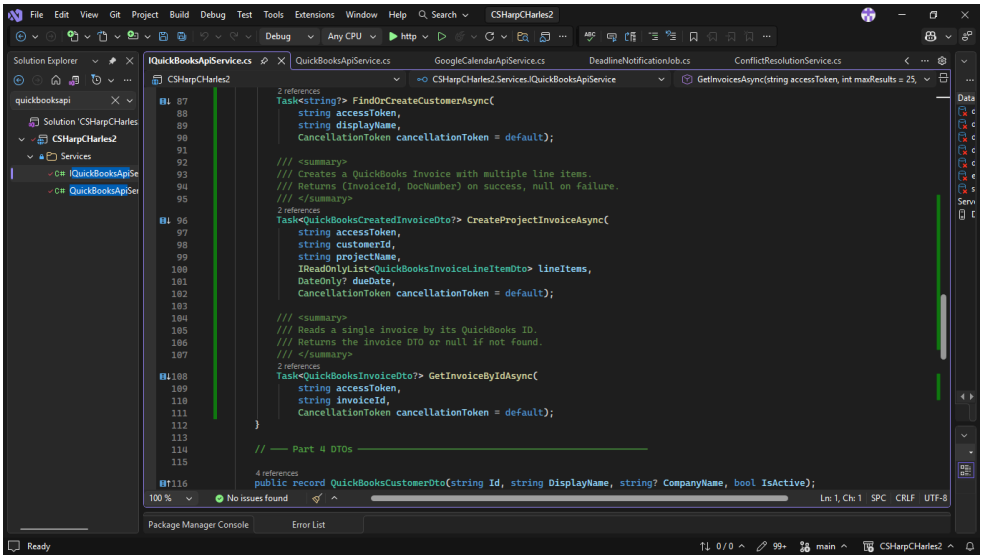
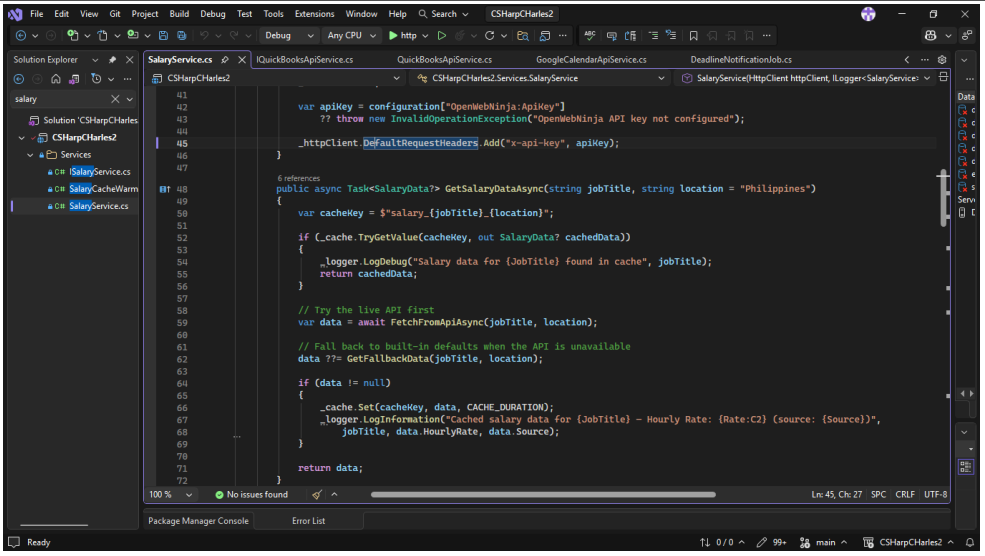
Google Calendar API — Fetch Calendar Events

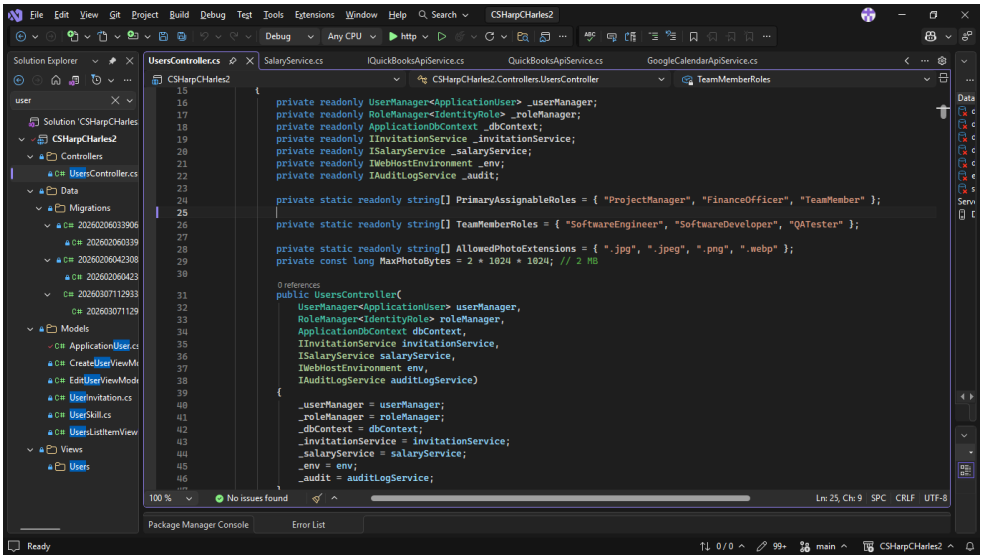
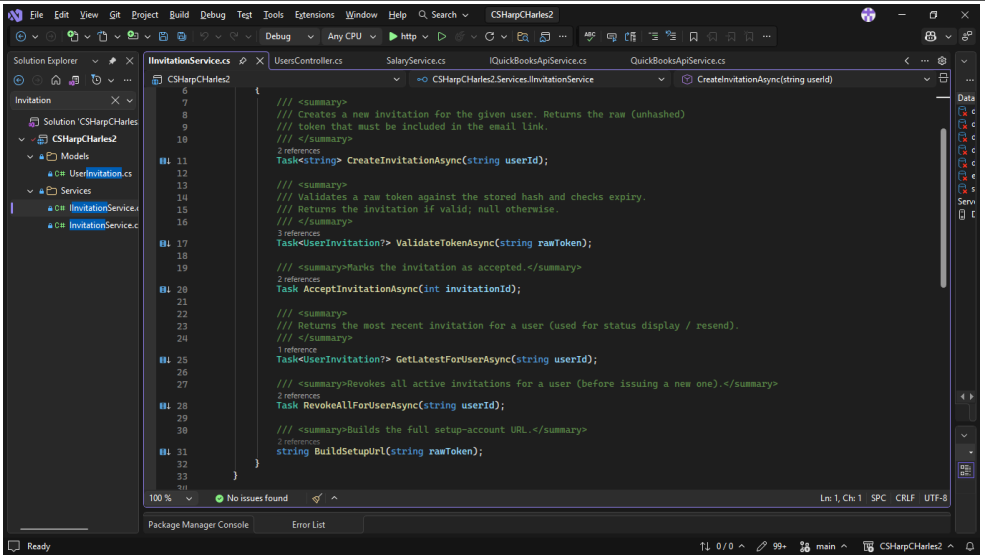
Calls POST /calendar/v3/calendars/{calendarId}/events to push a new event outbound to the user's Google Calendar. The request body is a JSON object containing the event title, start/end dateTime in RFC 3339 format, an optional description, and an optional attendees array of email objects. The created event's id, link, and times are parsed from the response and returned as a CalendarEventDto. Used when a project deadline or milestone is recorded in the system — it is automatically created as a calendar event for the assigned team member.

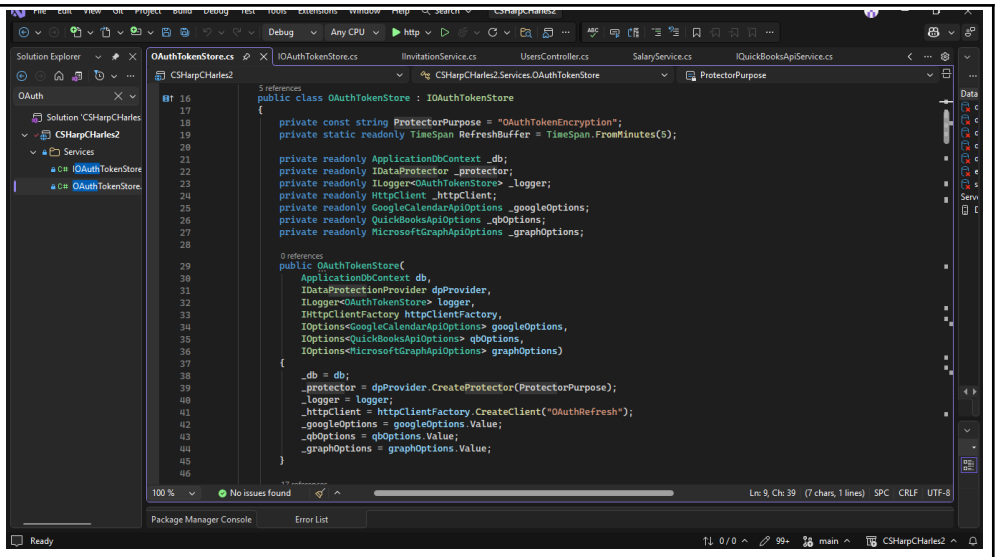


QuickBooks Online API — Query Invoices

Calls GET /v3/company/{companyId}/query on the QuickBooks Online API v3 using IQL (Intuit Query Language) — e.g., select * from Invoice maxresults 25. A Bearer OAuth token is attached, and minorversion=75 is appended for API compatibility. The JSON response is parsed from QueryResponse.Invoice[], extracting id, document number, total amount, balance, currency, due date, and customer name per invoice. GetAllInvoicesAsync extends this with cursor-based pagination — repeatedly fetching pages of 100 (startposition offset) until a partial

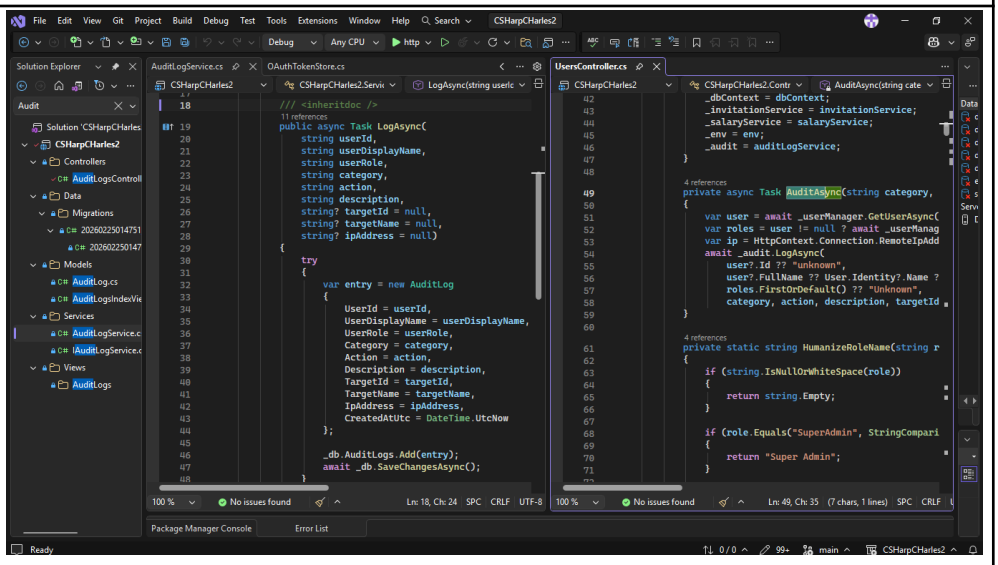
	page signals the last batch. Used in the Budget module's Receivables section and the QuickBooks integration panel in Admin Integrations.
	 **QuickBooks Online API — Create Invoice** When a project transitions to "Pending Payment" status, QuickBooksInvoiceService posts a new invoice to QuickBooks Online via POST /v3/company/{companyId}/invoice. The invoice line items are built from the project's completed tasks, the customer is mapped from the project's client name, and the due date is set from the project's end date. The returned QuickBooks invoice ID and document number are stored on the project record. A background job (QuickBooksInvoiceSyncJob) then periodically syncs the invoice status back — automatically marking the project as Completed when the invoice is marked Paid in QuickBooks.
	 **OpenWebNinja Job Salary Data API** Calls GET https://api.openwebninja.com/v1/job-salary-data/job-salary?job_title={title}&location={location} with an x-api-key header (loaded from appsettings.json). The response is deserialized into OpenWebNinjaResponse, extracting MinSalary and MaxSalary to compute a median annual salary, then dividing by 2,080 work hours/year to produce an hourly rate. Results are cached in IMemoryCache for 24 hours per job title + location key to avoid redundant API calls. If the API is unavailable or quota-exhausted, the service falls back to a hardcoded Philippine market salary table (13 roles, e.g., Software Engineer ₱480k–₱960k/yr). The

	hourly rate feeds directly into WeeklyLaborCostService, which accrues 40 hrs/week of labor cost onto each project's BudgetSpent field.
SECURITY FEATURES	 **Role-Based Access Control (RBAC)** Every controller is decorated with [Authorize] or [Authorize(Roles = "...")] at the class level. UsersController and AuditLogsController are locked to SuperAdmin only. BudgetController admits SuperAdmin, FinanceOfficer, and ProjectManager. Individual actions add further inline role checks (e.g., only SuperAdmin/FinanceOfficer may approve proposals; TeamMembers cannot see the lock banner). This prevents privilege escalation even if a user manually crafts a URL — the server re-validates role membership on every request through ASP.NET Core's [Authorize] middleware pipeline.
	 **Secure User Invitation Tokens** When a SuperAdmin creates a new user, InvitationService generates a 256-bit cryptographically random token using RandomNumberGenerator.GetBytes(32) and encodes it as URL-safe Base64 (RFC 4648). Only the SHA-256 hash of the token is stored in the database — the raw token is never persisted. When the user clicks the invitation link, ValidateTokenAsync re-hashes the submitted token and compares it to the stored hash. It additionally rejects the token if it has already been accepted (AcceptedAtUtc is set), if it is revoked (IsRevoked = true), or if it is past its 24-hour expiry. Any pre-existing active invitations for the same user are revoked before a new one is issued, preventing token accumulation.



OAuth Token Encryption at Rest

OAuth access tokens and refresh tokens (Google Calendar, QuickBooks) are never stored in plaintext. Before saving, each token is encrypted using ASP.NET Core Data Protection (IDataProtector with purpose string "OAuthTokenEncryption"). The keys are persisted to the filesystem (keys) and scoped to the application name "CSharpCharles", preventing tokens from being decrypted by a different application even on the same machine. On retrieval, decryption failure is caught and logged as a warning, returning null rather than crashing or exposing a partial token. Tokens are also checked against a 5-minute expiry buffer and silently refreshed before use.



Audit Logging

Every privileged action — user creation, role changes, budget approvals, project edits, report exports, integrations — writes an AuditLog record containing the acting user's ID, display name, role, IP address, UTC timestamp, action category, action name, and a human-readable description with the target entity. AuditLogService swallows all exceptions internally so a logging failure never interrupts the primary operation. The audit log is viewable only by SuperAdmins via AuditLogsController with filtering by role, category, action, date range, and free-text search — providing a full tamper-evident trail for accountability and incident investigation.

Date Submitted:	March 11, 2026

Teacher's Feedback	_____ _____ _____ _____ _____ _____ _____ _____ Student's Signature (after feedbacking) Teacher's Signature